

IES ARCIPRESTE DE HITA

Ciclo Formativo de Grado Superior — Desarrollo de Aplicaciones Web



GESTOR LIBRVM

Sistema de Gestión de Préstamos de Biblioteca Escolar

Alumnos:	David Martín y Andy Modoran
Tutor:	Aaron Montalvo
Asignatura:	Programación de aplicaciones utilizando frameworks
Fecha:	31 de Mayo de 2026
Versión:	1.0 — Versión Final

ÍNDICE DE CONTENIDOS

Tabla de contenido

ÍNDICE DE CONTENIDOS.....	2
1. INTRODUCCIÓN.....	3
1.1. Descripción del proyecto.....	3
1.2. Justificación.....	3
1.3. Ámbito del proyecto.....	3
2. OBJETIVOS.....	4
2.1. Objetivo General.....	4
2.2. Objetivos Específicos.....	4
3. ANÁLISIS DEL SISTEMA.....	5
3.1. Especificación de Requisitos.....	5
3.1.1. Requisitos Funcionales (RF).....	5
3.1.2. Requisitos No Funcionales (RNF).....	5
3.2. Identificación de Actores.....	6
3.3. Planificación y Costes.....	6
3.3.1. Planificación Temporal.....	6
3.3.2. Estimación de Costes.....	6
4. DISEÑO DEL SISTEMA.....	7
4.1. Arquitectura Tecnológica.....	7
4.1.1. Patrón MVC en Laravel.....	7
4.1.2. Infraestructura Docker.....	7
4.2. Diseño de Datos.....	7
4.2.1. Modelo E/R (Entidad-Relación).....	7
4.2.2. Estructura de Tablas.....	7
4.3. Diseño de Interfaz (UI/UX).....	9
4.3.1. Interfaz Pública (Catálogo).....	9
4.3.2. Interfaz Privada (Panel de Administración).....	9
5. IMPLEMENTACIÓN Y CODIFICACIÓN.....	10
5.1. Diccionario y Modelos de Datos.....	10
5.2. Backend.....	10
5.2.1. Controladores.....	10
5.2.2. Lógica de Negocio Clave.....	10
5.2.3. Seguridad y Roles.....	11
5.2.4. Rutas (routes/web.php).....	11
5.3. Frontend.....	11
5.4. Guía de Estilos.....	11
6. EVALUACIÓN FINAL.....	12

6.1. Casos de Uso Implementados.....	12
6.2. Arquitectura Final del Proyecto.....	12
6.3. Cumplimiento de Objetivos y Plazos.....	12
6.4. Validación y Cumplimiento Técnico.....	13
7. RESULTADO DE LAS PRUEBAS.....	14
7.1. Informe de Resultados.....	14
7.1.1. Configuración del Entorno de Pruebas.....	14
7.1.2. Resultados de Tests Automatizados.....	14
7.2. Verificación de Funcionalidades.....	14
8. REGISTRO DE INCIDENCIAS.....	16
9. REGISTRO DE CAMBIOS.....	17
10. DESPLIEGUE DEL PROYECTO.....	18
10.1. Despliegue en Servidor (Incluyendo Base de Datos).....	18
10.1.1. Pasos de Despliegue.....	18
10.1.2. Servicios y Puertos.....	18
10.1.3. Credenciales Iniciales (Seeders).....	18
10.2. Conexión en Cliente.....	18
11. MANUAL DE USO.....	20
11.1. Manual para Usuarios (Bibliotecarios).....	20
11.1.1. Iniciar Sesión.....	20
11.1.2. Gestión de Libros.....	20
11.1.3. Gestión de Alumnos.....	21
11.1.4. Gestión de Préstamos.....	22
11.1.5. Módulo de Morosos.....	23
11.2. Manual para Administradores.....	23
11.2.1. Gestión de Usuarios del Sistema.....	23
11.2.2. Auditoría — Historial de Movimientos.....	24
11.2.3. Gestión del Perfil.....	25
12. CONCLUSIONES.....	26
12.1. Síntesis de Resultados.....	26
12.2. Líneas Futuras de Trabajo.....	26
BIBLIOGRAFÍA Y REFERENCIAS.....	27

1. INTRODUCCIÓN

1.1. Descripción del proyecto

El proyecto Gestor Librvm consiste en el desarrollo de una aplicación web completa para la administración de bibliotecas escolares. El sistema permite digitalizar el inventario de libros, gestionar la base de datos de alumnos y controlar el flujo de préstamos y devoluciones de manera centralizada.

A diferencia de las soluciones tradicionales basadas en hojas de cálculo, este software ofrece una interfaz moderna y accesible vía navegador, garantizando la integridad de los datos y facilitando la labor del personal bibliotecario.

1.2. Justificación

La elección de desarrollar este sistema responde a la necesidad real de modernizar la gestión de recursos en centros educativos, donde el control manual suele derivar en pérdidas de material y errores administrativos.

Desde una perspectiva académica, este proyecto sirve para demostrar la competencia en el desarrollo Full Stack moderno. Se ha optado por utilizar Laravel como framework de backend y Bootstrap 5 para el frontend, tecnologías que representan el estándar actual en la industria, permitiendo crear una aplicación escalable, segura y mantenible.

1.3. Ámbito del proyecto

El sistema está diseñado para ser desplegado en la intranet del centro educativo. Sus funcionalidades abarcan desde la gestión del catálogo (altas y bajas de libros) hasta la validación de usuarios y el seguimiento de los plazos de devolución, diferenciando claramente entre un área de administración privada y un catálogo de acceso público.

2. OBJETIVOS

2.1. Objetivo General

Desarrollar una aplicación web robusta y segura para la gestión integral de una biblioteca escolar, que permita automatizar el control de préstamos, el inventario de libros y la gestión de usuarios, garantizando la integridad de los datos mediante un sistema de auditoría.

2.2. Objetivos Específicos

- **Arquitectura Robusta:** Implementar el backend utilizando el framework Laravel 12, aprovechando sus capacidades de enrutamiento, seguridad y ORM (Eloquent).
- **Interfaz Adaptativa:** Diseñar un frontend intuitivo y responsive mediante Bootstrap 5 y Blade, asegurando una experiencia de usuario óptima en todos los dispositivos.
- **Persistencia de Datos:** Gestionar la base de datos relacional (MariaDB) mediante el sistema de Migraciones de Laravel, con control de versiones sobre la estructura de información.
- **Gestión de Usuarios:** Implementar un sistema de autenticación seguro con roles diferenciados (Administrador vs. Bibliotecario) utilizando las librerías nativas del framework.
- **Automatización de Pruebas:** Asegurar la fiabilidad del código mediante pruebas automatizadas (Feature Tests y Unit Tests) con PHPUnit.
- **Despliegue Contenerizado:** Configurar el entorno de producción y desarrollo utilizando Docker y Docker Compose, eliminando los problemas de compatibilidad.
- **Gestión de Inventario:** Permitir el alta, baja y modificación de libros, con control de stock automático que impida prestar libros no disponibles.
- **Control de Préstamos:** Registrar la salida y entrada de libros, calculando automáticamente la disponibilidad en tiempo real.
- **Gestión de Incidencias:** Implementar un sistema para identificar y gestionar alumnos morosos o con devoluciones pendientes.
- **Auditoría:** Registrar en una tabla de movimientos todas las acciones críticas (borrados, altas, devoluciones) indicando usuario y fecha.

3. ANÁLISIS DEL SISTEMA

3.1. Especificación de Requisitos

El sistema cuenta con dos actores claramente diferenciados:

- **Administrador:** Tiene acceso completo al sistema, incluyendo la Gestión de Usuarios (crear/editar bibliotecarios) y el módulo de Auditoría (visualizar historial de acciones).
- **Bibliotecario (Usuario):** Es el encargado del día a día. Puede gestionar libros, alumnos y realizar préstamos/devoluciones, pero no tiene acceso a la configuración global ni a los logs del sistema.

3.1.1. Requisitos Funcionales (RF)

Código	Nombre	Descripción
RF-01	Gestión de Roles	Política de acceso basada en roles: Administrador (acceso total) y Bibliotecario (acceso operativo).
RF-02	Catálogo Público	Buscador público accesible sin autenticación para filtrar libros por título o autor y ver disponibilidad.
RF-03	Gestión de Inventario	CRUD completo de libros. Validación de unicidad de ISBN. Bloqueo de borrado si hay préstamos activos.
RF-04	Gestión de Alumnos	Alta, baja y modificación de alumnos. Validación del formato NIE/DNI.
RF-05	Gestión de Préstamos	Verificación automática de stock (>0). Asignación del libro y decremento del inventario.
RF-06	Gestión de Devoluciones	Cierre de préstamo activo, liberación del ejemplar e incremento de stock.
RF-07	Módulo de Morosos	Listado de alumnos con préstamos vencidos o retrasados para su seguimiento.
RF-08	Auditoría	Registro automático de todas las acciones críticas: quién las realizó y cuándo.
RF-09	Gestión de Usuarios	Administración de cuentas del sistema (solo rol Admin). Alta, edición y baja de usuarios.

3.1.2. Requisitos No Funcionales (RNF)

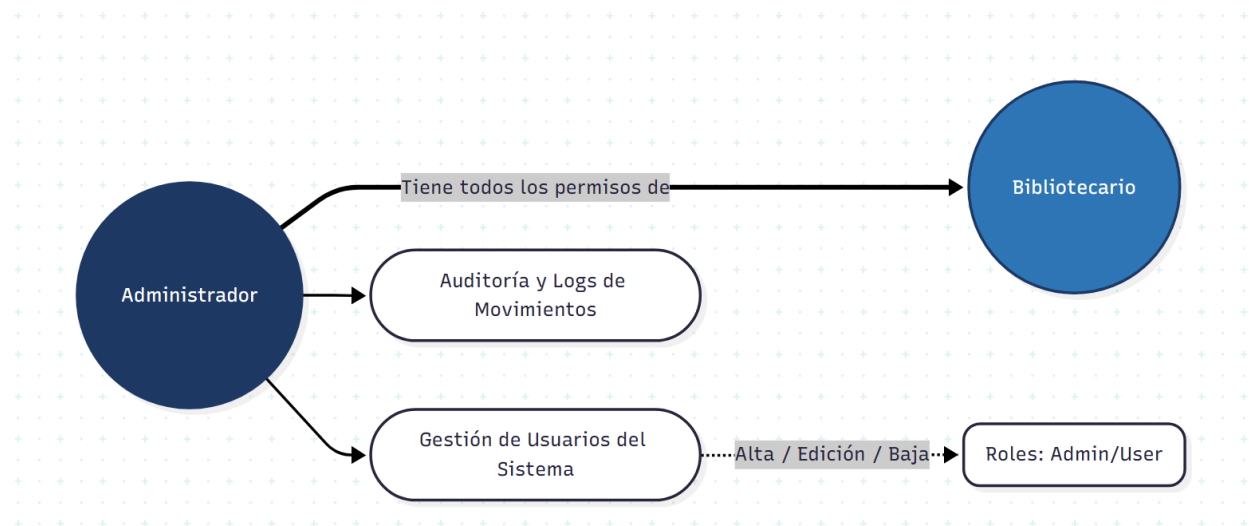
Código	Nombre	Descripción
RNF-01	Stack Tecnológico	Backend en Laravel 12, PHP 8.2, dependencias gestionadas con Composer.
RNF-02	Interfaz de Usuario	Bootstrap 5 + Blade. Interfaz responsive adaptable a dispositivos móviles y escritorio.
RNF-03	Persistencia	Base de datos MariaDB. ORM Eloquent para prevenir inyecciones SQL. Contraseñas con Bcrypt.
RNF-04	Despliegue	Entorno containerizado con Docker y Docker Compose. Independiente del sistema operativo.
RNF-05	Rendimiento	Tiempo de respuesta < 2 segundos en operaciones CRUD bajo carga normal.

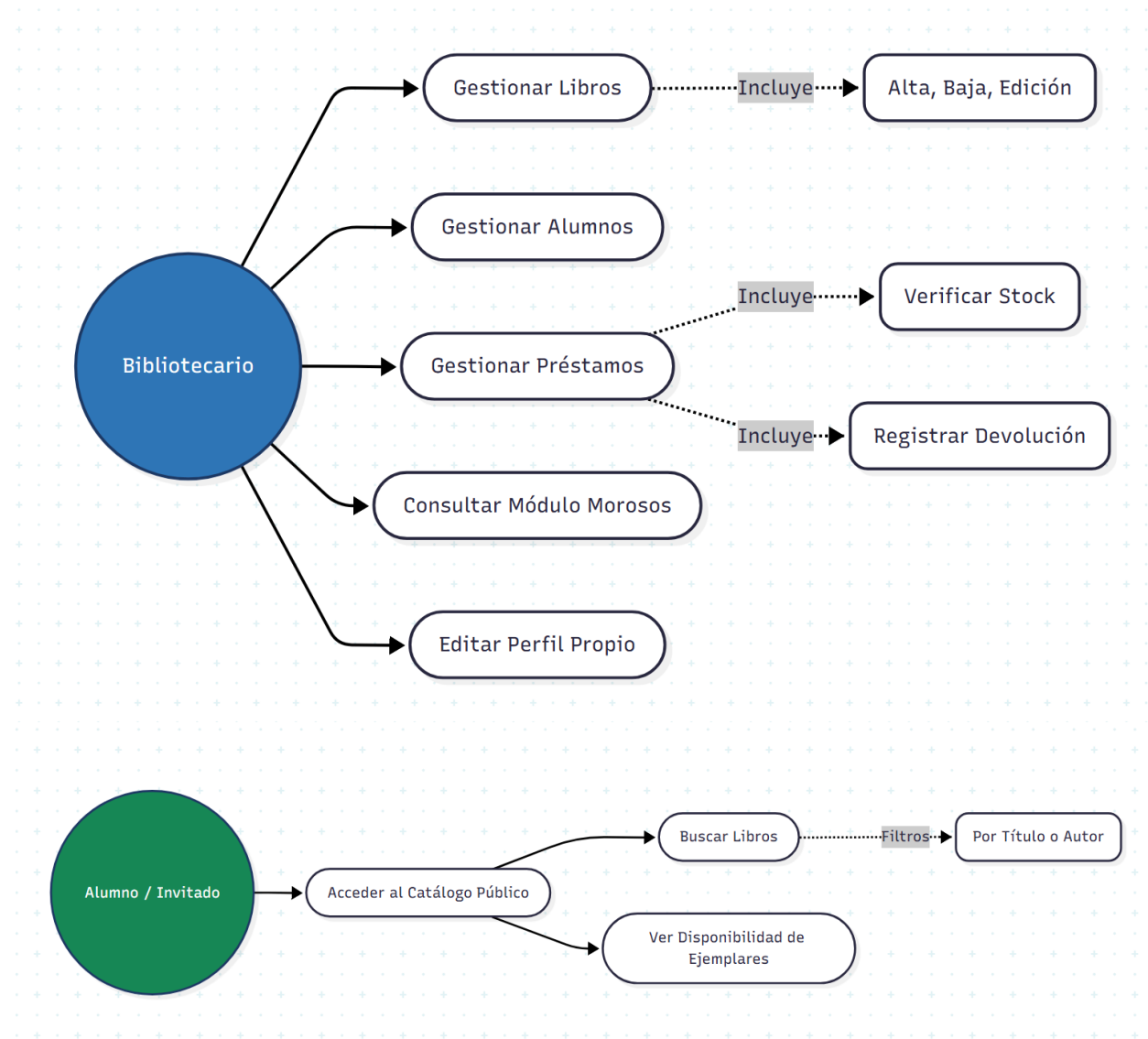
Código	Nombre	Descripción
RNF-06	Seguridad	Protección CSRF en todos los formularios. Middleware de autenticación en rutas privadas.

3.2. Identificación de Actores

Se han identificado dos actores principales que interactúan con el sistema:

- **Administrador:** Actor principal con privilegios totales. Gestiona el catálogo, préstamos, alumnos, usuarios del sistema y visualiza logs de auditoría.
- **Bibliotecario (Usuario):** Actor operativo con acceso a gestión de libros, alumnos y préstamos del día a día. No accede a configuración global ni logs.
- **Alumno (Usuario Invitado):** Actor secundario con permisos de solo lectura. Accede al catálogo público para verificar disponibilidad de libros.





3.3. Planificación y Costes

3.3.1. Planificación Temporal

Fase	Inicio	Fin	Estado
Análisis y Requisitos	21/01/2026	23/01/2026	Completado
Diseño de BD y Arquitectura	23/01/2026	25/01/2026	Completado
Implementación Backend	25/01/2026	05/02/2026	Completado
Implementación Frontend	05/02/2026	15/02/2026	Completado
Pruebas Automatizadas	15/02/2026	20/02/2026	Completado
Despliegue y Documentación	20/02/2026	31/05/2026	Completado

3.3.2. Estimación de Costes

Concepto	Horas	Coste/hora	Total
Desarrollo Backend (Laravel)	60h	0 € (prácticas)	0 €

Concepto	Horas	Coste/hora	Total
Desarrollo Frontend (Blade/Bootstrap)	40h	0 € (prácticas)	0 €
Configuración Docker	10h	0 € (prácticas)	0 €
Testing y QA	15h	0 € (prácticas)	0 €
Documentación	20h	0 € (prácticas)	0 €
Licencias de software (todo open source)	-	-	0 €
TOTAL	145h	-	0 €

4. DISEÑO DEL SISTEMA

4.1. Arquitectura Tecnológica

4.1.1. Patrón MVC en Laravel

La aplicación sigue estrictamente el patrón de arquitectura Modelo-Vista-Controlador (MVC), estándar en el desarrollo con Laravel. El flujo de una petición típica es:

1. **Ruta (Route):** El archivo routes/web.php recibe la petición HTTP.
2. **Middleware:** Verifica si el usuario está autenticado y tiene los permisos necesarios.
3. **Controlador (Controller):** El controlador correspondiente recibe los datos, los valida y ejecuta la lógica de negocio.
4. **Modelo (Model):** Se comunica con la base de datos mediante Eloquent ORM para guardar o recuperar datos.
5. **Vista (View):** Retorna una respuesta visual (vista Blade) al usuario con el resultado de la operación.

4.1.2. Infraestructura Docker

El entorno está compuesto por los siguientes servicios orquestados con Docker Compose:

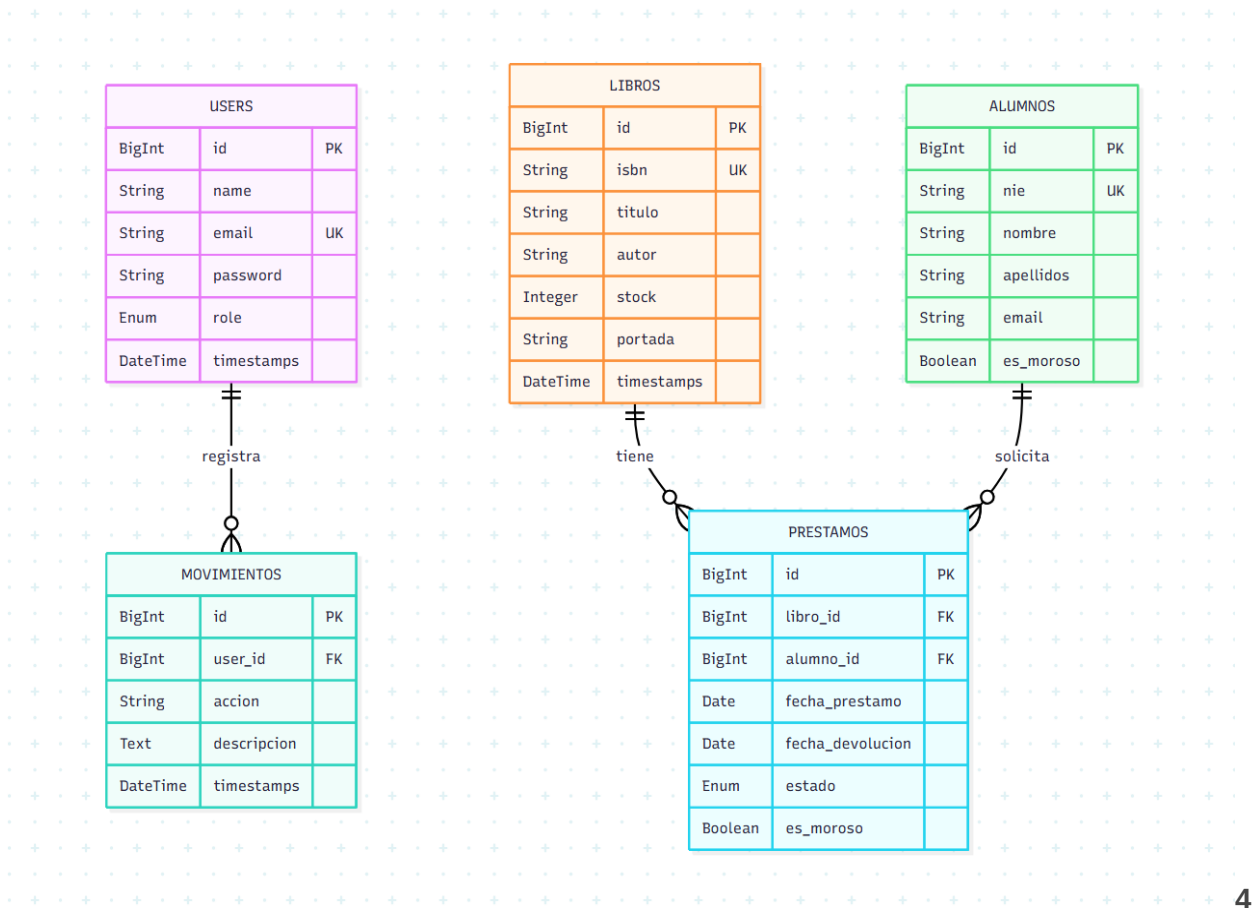
Servicio	Imagen	Función
app	PHP 8.2 + Apache	Servidor de aplicación Laravel. Ejecuta el código PHP.
db	MariaDB 10.6	Base de datos relacional. Persiste todos los datos del sistema.
phpmyadmin	phpMyAdmin	Interfaz web para gestión visual de la base de datos.

4.2. Diseño de Datos

4.2.1. Modelo E/R (Entidad-Relación)

El sistema se estructura en torno a cinco entidades principales con las siguientes relaciones:

- **User (1) → (N) Movimientos:** Un User puede registrar múltiples Movimientos (auditoría).
- **Libro (1) → (N) Prestamos:** Un Libro puede tener múltiples Prestamos a lo largo del tiempo.
- **Alumno (1) → (N) Prestamos:** Un Alumno puede tener múltiples Prestamos activos o históricos.
- **Prestamo (N) → (1) Libro + (1) Alumno:** Un Prestamo pertenece exactamente a un Libro y a un Alumno.



4

.2.2. Estructura de Tablas

Tabla: users (Usuarios del sistema)

Campo	Tipo	Descripción
id	BigInt PK	Identificador único autoincremental.
name	String	Nombre del usuario del sistema.
email	String Unique	Correo electrónico para login.
password	String	Contraseña cifrada con Bcrypt.
role	Enum	'admin' o 'user'. Define los privilegios del usuario.
timestamps	DateTime	created_at y updated_at automáticos.

Tabla: libros (Inventario de la biblioteca)

Campo	Tipo	Descripción
id	BigInt PK	Identificador único.
isbn	String Unique	Código ISBN-13 validado. No puede repetirse.
titulo	String	Título completo de la obra.
autor	String	Autor o autores principales.
stock	Integer	Cantidad de ejemplares disponibles para préstamo.
portada	String Nullable	Ruta a la imagen de portada del libro.

Campo	Tipo	Descripción
timestamps	DateTime	created_at y updated_at automáticos.

Tabla: alumnos (Prestatarios)

Campo	Tipo	Descripción
id	BigInt PK	Identificador único.
nie	String Unique	Documento de identidad del alumno (validado).
nombre	String	Nombre del alumno.
apellidos	String	Apellidos del alumno.
email	String Nullable	Email para futuras notificaciones.
es_moroso	Boolean	Indica si el alumno tiene préstamos vencidos.

Tabla: prestamos (Transacciones de préstamo)

Campo	Tipo	Descripción
id	BigInt PK	Identificador único.
libro_id	BigInt FK	Referencia a la tabla libros.
alumno_id	BigInt FK	Referencia a la tabla alumnos.
fecha_prestamo	Date	Fecha de salida del libro.
fecha_devolucion	Date Nullable	Fecha de retorno. NULL = libro aún prestado.
estado	Enum	'activo', 'devuelto' o 'retrasado'.
es_moroso	Boolean	Flag calculado según fecha de devolución esperada.

Tabla: movimientos (Auditoría)

Campo	Tipo	Descripción
id	BigInt PK	Identificador único.
user_id	BigInt FK	Usuario que realizó la acción.
accion	String	Tipo de acción: 'Préstamo', 'Devolución', 'Alta libro', etc.
descripcion	Text	Descripción detallada del movimiento.
timestamps	DateTime	Fecha y hora exacta de la acción.

4.3. Diseño de Interfaz (UI/UX)

4.3.1. Interfaz Pública (Catálogo)

- **Navbar:** Logotipo y botón de Acceso Administrador.
- **Hero Section:** Buscador grande centrado con filtros por Título y Autor.

- **Grid de Resultados:** Tarjetas Bootstrap mostrando portada, título y etiqueta de disponibilidad.

4.3.2. Interfaz Privada (Panel de Administración)

- **Sidebar/Dashboard:** Menú lateral para navegar entre Libros, Alumnos, Préstamos, Morosos y Logs.
- **Tablas de Datos:** Listados paginados con botones de acción (Editar, Borrar).
- **Modales:** Formularios emergentes para editar registros sin recargar la página.
- **Alertas de Feedback:** Mensajes Toast de confirmación tras cada operación.

5. IMPLEMENTACIÓN Y CODIFICACIÓN

5.1. Diccionario y Modelos de Datos

Los modelos Eloquent implementados en app/Models/ son:

Modelo	Tabla BD	Relaciones principales
User	users	hasMany(Movimiento). Atributo role para control de acceso.
Libro	libros	hasMany(Prestamo). Control de stock automático.
Alumno	alumnos	hasMany(Prestamo). Flag es_moroso calculado.
Prestamo	prestamos	belongsTo(Libro), belongsTo(Alumno). Modelo central.
Movimiento	movimientos	belongsTo(User). Registro de auditoría del sistema.

5.2. Backend

5.2.1. Controladores

Los controladores principales implementados en app/Http/Controllers/ son:

- **LibroController:** Gestión de libros (CRUD completo, bloqueo de borrado con préstamos activos, validación ISBN único).
- **AlumnoController:** Gestión de alumnos (CRUD, validación NIE/DNI, marcado automático de morosos).
- **PrestamoController:** Lógica transaccional de préstamos: verificación de stock, creación, devolución y borrado con ajuste de inventario.
- **MorosoController:** Lista de alumnos con préstamos retrasados para seguimiento.
- **UserController:** Gestión de usuarios del sistema (solo Admin): creación, edición y baja de cuentas.
- **AdminController:** Visualización del registro de auditoría (solo Admin).

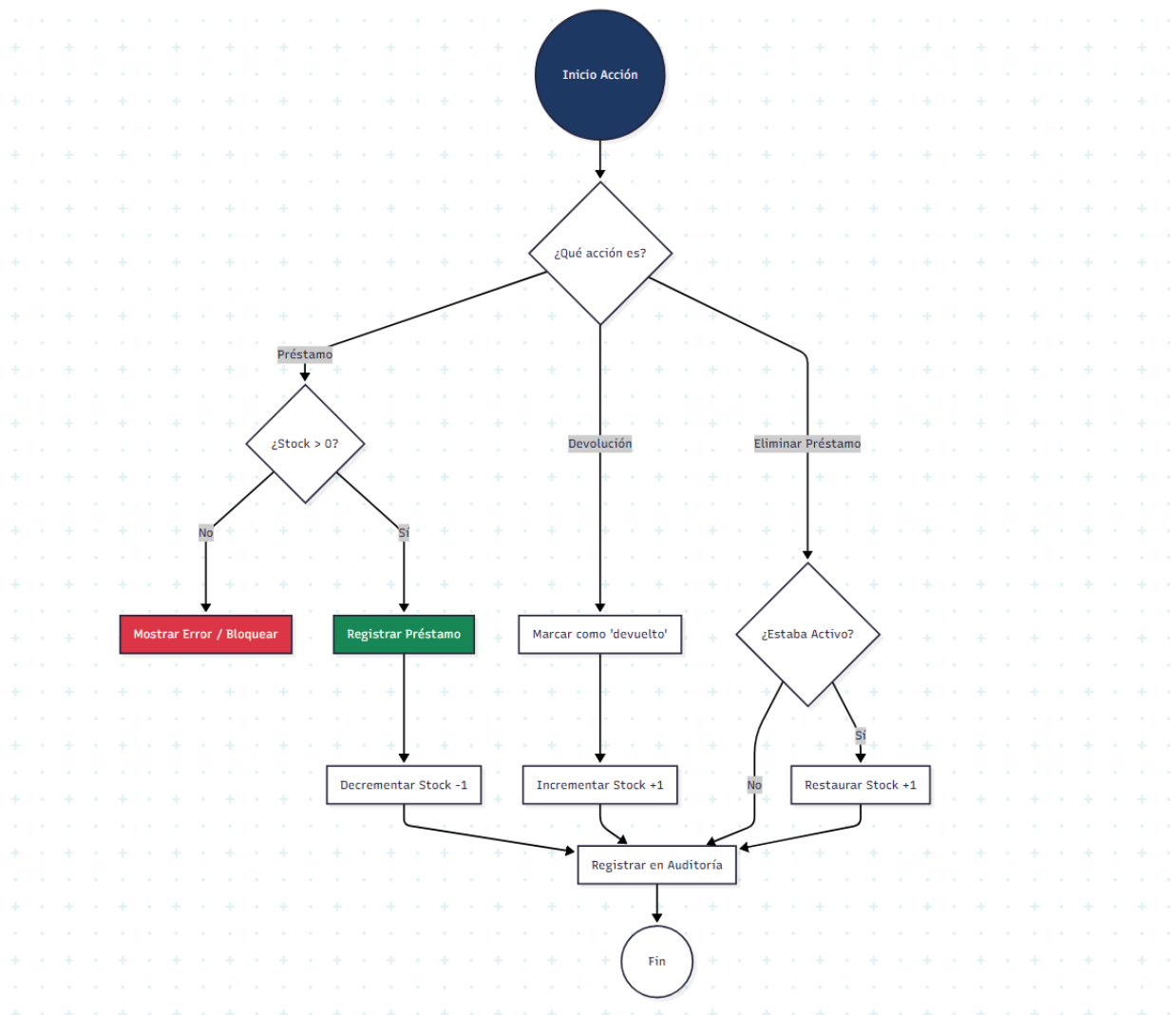
5.2.2. Lógica de Negocio Clave

A. Gestión Inteligente de Stock (PrestamoController):

- **Préstamo:** Al crear un préstamo (store): verifica stock > 0, crea el préstamo y decrementa el stock del libro.
- **Devolución:** Al devolver (devolver): marca fecha de devolución y estado 'devuelto', incrementa el stock.
- **Eliminación:** Al eliminar (destroy): si el préstamo era activo, devuelve el stock al libro para evitar descuadres.

B. Integridad Referencial (LibroController): Antes de eliminar un libro, se comprueba si existen préstamos asociados. Si existen, se bloquea la eliminación mostrando un mensaje de error al usuario.

C. Sistema de Auditoría: En cada acción crítica de los controladores se invoca al modelo Movimiento para registrar automáticamente el usuario, la acción y una descripción detallada.



5.2.3. Seguridad y Roles

La seguridad se implementa en dos capas:

6. **Autenticación:** Laravel Breeze gestiona el registro, login y recuperación de contraseñas con protección CSRF.
7. **Autorización:** Las rutas privadas están protegidas con el middleware 'auth'. Las funciones de administración verifican el rol con `Auth::user()->role !== 'admin'` devolviendo 403 si no corresponde.

5.2.4. Rutas (routes/web.php)

La estructura de rutas está organizada en grupos:

- **Públicas (sin middleware):** Rutas públicas: catálogo (/), búsqueda.
- **Auth (Breeze):** Rutas de autenticación gestionadas por Breeze (login, register, logout).
- **Middleware 'auth':** Rutas de gestión: /libros, /alumnos, /prestamos, /morosos, /profile.
- **Solo Admin:** Rutas restringidas: /users, /admin/movimientos.

5.3. Frontend

La interfaz se ha construido con el motor de plantillas Blade, aprovechando la herencia de layouts:

- **app.blade.php (Layout principal):** Estructura base HTML con navbar dinámica según el rol del usuario (Admin muestra menús adicionales).
- **guest.blade.php (Layout invitado):** Layout simplificado para las páginas de login y registro.
- **Vistas de módulo:** Vistas organizadas por módulo: libros/, alumnos/, prestamos/, morosos/, users/, admin/.
- **Sistema de alertas:** Feedback visual con alertas de Bootstrap (verdes para éxito, rojas para error).
- **Modales Bootstrap:** Formularios modales para edición sin recarga de página.

5.4. Guía de Estilos

Elemento	Valor	Aplicación
Color primario	#1F3864 (azul oscuro)	Encabezados, sidebar, botones principales.
Color secundario	#2E75B6 (azul medio)	Botones secundarios, bordes de tabla.
Color éxito	#198754 (verde)	Alertas de éxito, badges 'Disponible'.
Color peligro	#DC3545 (rojo)	Alertas de error, badges 'Agotado'.
Fuente principal	Roboto / sans-serif	Cuerpo de texto, menús.
Framework CSS	Bootstrap 5 (CDN)	Grid, componentes, utilidades responsive.
Iconos	Bootstrap Icons	Iconografía en botones y menús.

6. EVALUACIÓN FINAL

6.1. Casos de Uso Implementados

ID	Caso de Uso	Actor	Estado
CU-01	Iniciar sesión con email y contraseña	Admin / User	Implementado
CU-02	Cerrar sesión	Admin / User	Implementado
CU-03	Buscar libros en catálogo público	Invitado	Implementado
CU-04	Ver disponibilidad de libro	Invitado	Implementado
CU-05	Crear nuevo libro (con ISBN)	Admin / User	Implementado
CU-06	Editar datos de un libro	Admin / User	Implementado
CU-07	Eliminar libro (con verificación)	Admin / User	Implementado
CU-08	Dar de alta un alumno	Admin / User	Implementado
CU-09	Editar datos de un alumno	Admin / User	Implementado
CU-10	Dar de baja un alumno	Admin / User	Implementado
CU-11	Realizar un préstamo	Admin / User	Implementado
CU-12	Registrar devolución de libro	Admin / User	Implementado
CU-13	Ver lista de morosos	Admin / User	Implementado
CU-14	Ver historial de auditoría (logs)	Solo Admin	Implementado
CU-15	Crear nuevo usuario del sistema	Solo Admin	Implementado
CU-16	Editar usuario del sistema	Solo Admin	Implementado
CU-17	Eliminar usuario del sistema	Solo Admin	Implementado
CU-18	Editar perfil propio	Admin / User	Implementado

6.2. Arquitectura Final del Proyecto

La arquitectura final del proyecto respeta el patrón MVC implementado con Laravel 12 sobre PHP 8.2, dockerizado con tres servicios: la aplicación (Apache + PHP), la base de datos (MariaDB) y phpMyAdmin. La estructura de carpetas sigue el estándar de Laravel:

- **Backend:** app/Http/Controllers/ — Lógica de negocio y controladores.
- **Backend:** app/Models/ — Modelos Eloquent y relaciones.
- **Frontend:** resources/views/ — Plantillas Blade organizadas por módulo.
- **Enrutamiento:** routes/web.php — Definición de rutas con grupos por middleware.
- **Base de Datos:** database/migrations/ — Estructura de la BD versionada.
- **Base de Datos:** database/seeders/ — Datos iniciales (admin, maria, libros y alumnos de prueba).
- **Testing:** tests/Feature/ — Suite de pruebas automatizadas.

6.3. Cumplimiento de Objetivos y Plazos

Objetivo	Plazo Estimado	Resultado
Implementar CRUD de libros con validación ISBN	Semana 1-2	Completado
Implementar CRUD de alumnos con validación NIE	Semana 1-2	Completado
Sistema de préstamos con control de stock	Semana 2-3	Completado
Módulo de morosos	Semana 3	Completado
Sistema de roles Admin / User	Semana 2	Completado
Auditoría de movimientos	Semana 3	Completado
Catálogo público	Semana 4	Completado
Despliegue con Docker Compose + deploy.sh	Semana 4-5	Completado
Suite de tests automatizados PHPUnit	Semana 4	Completado

6.4. Validación y Cumplimiento Técnico

Todos los requisitos funcionales y no funcionales definidos en el apartado 3.1 han sido implementados y validados. El sistema ha superado las siguientes verificaciones técnicas:

- **Seguridad:** Protección CSRF activa en todos los formularios del sistema.
- **Seguridad:** Middleware 'auth' aplicado correctamente en todas las rutas privadas.
- **Control de Acceso:** Verificación de rol 'admin' en controladores de usuario y auditoría, devolviendo 403 si se accede sin permisos.
- **Integridad de Inventario:** Stock nunca queda en valores negativos gracias a la verificación previa al préstamo.
- **Integridad Referencial:** No es posible eliminar un libro con préstamos activos en el historial.
- **Responsive Design:** Interfaces correctamente visualizadas en resoluciones de escritorio, tablet y móvil.
- **Calidad del Código:** El 100% de los tests automatizados (PHPUnit) han pasado correctamente.

7. RESULTADO DE LAS PRUEBAS

7.1. Informe de Resultados

La estrategia de pruebas combina tests automatizados (PHPUnit) para la lógica crítica y pruebas manuales para la interfaz de usuario.

7.1.1. Configuración del Entorno de Pruebas

- **Framework:** PHPUnit (integrado en Laravel).
- **Base de Datos:** SQLite en memoria (:memory:) para tests rápidos sin persistencia.
- **Configuración:** Archivo phpunit.xml con entorno de test aislado del de producción.

7.1.2. Resultados de Tests Automatizados

Test	Descripción	Resultado	Tiempo
PrestamoTest	Verifica creación de préstamo, descuento de stock, devolución, incremento y bloqueo con stock=0.	PASS	< 100ms
SecurityTest	Verifica que un usuario 'user' recibe 403 al acceder a /admin/logs.	PASS	< 50ms
LibroTest	Verifica CRUD básico de libros y validaciones de campos obligatorios.	PASS	< 80ms
AuthenticationTest	Verifica login con credenciales correctas e incorrectas.	PASS	< 60ms
RegistrationTest	Verifica el registro de nuevos usuarios con validación de datos.	PASS	< 70ms
TOTAL	Todos los tests de la suite	100% PASS	< 500ms

Comando para ejecutar los tests:

```
|docker-compose exec app php artisan test
```

7.2. Verificación de Funcionalidades

Funcionalidad	Método de Verificación	Resultado
Login correcto	Test automatizado + prueba manual	OK
Login incorrecto (error visible)	Prueba manual	OK
Crear libro con ISBN duplicado	Prueba manual (error esperado)	OK
Préstamo con stock suficiente	Test automatizado + prueba manual	OK
Préstamo con stock = 0 (bloqueado)	Test automatizado + prueba manual	OK
Devolución restaura stock	Test automatizado + prueba manual	OK
Eliminar libro con préstamos	Prueba manual	OK

Funcionalidad	Método de Verificación	Resultado
(bloqueado)		
Acceso a /admin/logs sin rol Admin (403)	Test automatizado + prueba manual	OK
Visualización responsive en móvil	Prueba manual (DevTools)	OK
Catálogo público sin login	Prueba manual	OK
Registros de auditoría tras acciones	Prueba manual (tabla movimientos)	OK

8. REGISTRO DE INCIDENCIAS

ID	Incidencia	Resolución	Fecha	Estado
I-01	Error 419 (CSRF expirado) al enviar formularios tras inactividad.	Se limpiaron cachés de configuración y sesión. Se ajustó el tiempo de sesión en .env.	22/01/2026	Resuelta
I-02	Stock de libros quedaba negativo al realizar borrado de préstamo activo.	Se añadió lógica condicional en PrestamoController@destroy para verificar estado antes de ajustar stock.	28/01/2026	Resuelta
I-03	Rutas de admin accesibles por usuarios sin rol 'admin' (sin 403).	Se añadió verificación de rol explícita en UserController y AdminController con abort(403).	02/02/2026	Resuelta
I-04	Migraciones fallaban al ejecutarse en Docker por permisos en SQLite.	Se cambió a MariaDB en el entorno Docker. Se actualizó .env y docker-compose.yml.	05/02/2026	Resuelta
I-05	Imágenes de portada no se mostraban por ruta incorrecta en producción.	Se utilizó asset() en vez de url() para generar rutas relativas correctas en Blade.	10/02/2026	Resuelta
I-06	Tests fallaban por conflicto de base de datos entre entorno test y desarrollo.	Se configuró phpunit.xml con DB_CONNECTION=sqlite y DB_DATABASE=:memory:.	18/02/2026	Resuelta

9. REGISTRO DE CAMBIOS

Ver.	Fecha	Cambio	Responsable
0.1	21/01/2026	Creación inicial del proyecto Laravel. Configuración Docker Compose.	David Martín
0.2	23/01/2026	Migraciones y seeders: tablas users, libros, alumnos, prestamos.	Andy Modoran
0.3	26/01/2026	Implementación CRUD completo de Libros y Alumnos.	David Martín
0.4	28/01/2026	Módulo de Préstamos con control de stock automático.	Andy Modoran
0.5	30/01/2026	Sistema de roles Admin/User. Middleware de autorización.	David Martín
0.6	02/02/2026	Módulo de Auditoría (tabla movimientos). AdminController.	Andy Modoran
0.7	05/02/2026	Módulo de Morosos. Cálculo automático de préstamos vencidos.	David Martín
0.8	10/02/2026	Catálogo público con buscador sin autenticación.	Andy Modoran
0.9	18/02/2026	Suite de tests automatizados (PrestamoTest, SecurityTest, LibroTest).	David Martín
1.0	31/05/2026	Versión final. Script deploy.sh. Optimización de caché y permisos.	David / Andy

10. DESPLIEGUE DEL PROYECTO

10.1. Despliegue en Servidor (Incluyendo Base de Datos)

El proyecto incluye un script de despliegue automático (deploy.sh) que automatiza todo el proceso. Los requisitos previos son: Docker Desktop instalado y Git Bash (en Windows) o terminal Linux/Mac.

10.1.1. Pasos de Despliegue

8. **Obtener el proyecto:** Descomprimir el archivo del proyecto en una carpeta local.
9. **Abrir terminal:** Abrir Git Bash (Windows) o Terminal (Linux/Mac) y navegar a la carpeta:
`cd /ruta/a/Final_DWES`
10. **Ejecutar deploy.sh:** Dar permisos de ejecución y ejecutar el script:
`bash deploy.sh`

El script realiza automáticamente los siguientes pasos:

- **Paso 1:** docker-compose up -d --build — Levanta y construye los contenedores Docker.
- **Paso 2:** composer install — Instala todas las dependencias PHP del proyecto.
- **Paso 3:** Copia src/.env.example a src/.env y genera la APP_KEY si no existe.
- **Paso 4:** sleep 10 — Espera a que MariaDB esté completamente disponible.
- **Paso 5:** php artisan migrate:fresh --seed --force — Crea todas las tablas y carga datos iniciales.
- **Paso 6:** Limpia cachés de configuración, rutas y vistas.
- **Paso 7:** chmod -R 777 storage bootstrap/cache — Ajusta permisos de escritura.

10.1.2. Servicios y Puertos

Servicio	Puerto	URL de acceso
Aplicación Laravel (Apache)	8090	http://localhost:8090
phpMyAdmin	8091	http://localhost:8091
MariaDB	3309	Acceso solo interno desde el contenedor app.

10.1.3. Credenciales Iniciales (Seeders)

Usuario	Email	Contraseña	Rol
Administrador	admin@test.com	password	admin
María García	maria@test.com	password	user

10.2. Conexión en Cliente

Una vez desplegado el servidor, el acceso desde el cliente es exclusivamente a través del navegador web, sin necesidad de instalar ningún software adicional.

11. **Acceso a la aplicación:** Abrir cualquier navegador web moderno (Chrome, Firefox, Edge).

12. **URL de acceso:** Navegar a: <http://localhost:8090> para la aplicación principal.
13. **Catálogo:** El catálogo público es accesible directamente sin autenticación.
14. **Panel privado:** Para el panel de administración, hacer clic en 'Acceso Administrador' e introducir las credenciales.

11. MANUAL DE USO

11.1. Manual para Usuarios (Bibliotecarios)

11.1.1. Iniciar Sesión

Para acceder al panel de gestión, el bibliotecario debe:

15. Abrir el navegador y acceder a <http://localhost:8090>.
16. Hacer clic en el botón 'Acceso Administrador' en la esquina superior derecha.
17. Introducir el email y contraseña proporcionados por el administrador.
18. Hacer clic en 'Iniciar Sesión'.

```
C:\Users\modor\Desktop\Final_DWES>docker-compose exec app php artisan make:model Libro
time="2026-01-21T13:39:59+01:00" level=warning msg="C:\\Users\\modor\\Desktop\\Final_DWES\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
INFO Model [app/Models/Libro.php] created successfully.

C:\Users\modor\Desktop\Final_DWES>docker-compose exec app php artisan make:model Alumno
time="2026-01-21T13:40:10+01:00" level=warning msg="C:\\Users\\modor\\Desktop\\Final_DWES\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
INFO Model [app/Models/Alumno.php] created successfully.

C:\Users\modor\Desktop\Final_DWES>docker-compose exec app php artisan make:model Prestamo
time="2026-01-21T13:40:20+01:00" level=warning msg="C:\\Users\\modor\\Desktop\\Final_DWES\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
INFO Model [app/Models/Prestamo.php] created successfully.

C:\Users\modor\Desktop\Final_DWES>docker-compose exec app php artisan make:controller LibroController --resource
time="2026-01-21T13:40:29+01:00" level=warning msg="C:\\Users\\modor\\Desktop\\Final_DWES\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
INFO Controller [app/Http/Controllers/LibroController.php] created successfully.

C:\Users\modor\Desktop\Final_DWES>docker-compose exec app php artisan make:controller AlumnoController --resource
time="2026-01-21T13:40:40+01:00" level=warning msg="C:\\Users\\modor\\Desktop\\Final_DWES\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
INFO Controller [app/Http/Controllers/AlumnoController.php] created successfully.

C:\Users\modor\Desktop\Final_DWES>docker-compose exec app php artisan make:controller PrestamoController --resource
time="2026-01-21T13:40:45+01:00" level=warning msg="C:\\Users\\modor\\Desktop\\Final_DWES\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
INFO Controller [app/Http/Controllers/PrestamoController.php] created successfully.

C:\Users\modor\Desktop\Final_DWES>
```

Pantalla de inicio - Catálogo público

```
C:\Windows\System32\cmd.exe X + v
0001_01_01_000001_create_cache_table ..... 101.21ms DONE
0001_01_01_000002_create_jobs_table ..... 182.05ms DONE

C:\Users\modor\Desktop\Final_DWES>docker-compose exec app php artisan make:migration create_libros_table
time="2026-01-21T13:33:48+01:00" level=warning msg="C:\\Users\\modor\\Desktop\\Final_DWES\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
INFO Migration [database/migrations/2026_01_21_123404_create_libros_table.php] created successfully.

C:\Users\modor\Desktop\Final_DWES>docker-compose exec app php artisan make:migration create_alumnos_table
time="2026-01-21T13:34:11+01:00" level=warning msg="C:\\Users\\modor\\Desktop\\Final_DWES\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
INFO Migration [database/migrations/2026_01_21_123423_create_alumnos_table.php] created successfully.

C:\Users\modor\Desktop\Final_DWES>docker-compose exec app php artisan make:migration create_prestamos_table
time="2026-01-21T13:34:34+01:00" level=warning msg="C:\\Users\\modor\\Desktop\\Final_DWES\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
INFO Migration [database/migrations/2026_01_21_123447_create_prestamos_table.php] created successfully.

C:\Users\modor\Desktop\Final_DWES>docker-compose exec app php artisan migrate
time="2026-01-21T13:38:31+01:00" level=warning msg="C:\\Users\\modor\\Desktop\\Final_DWES\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
INFO Preparing database.
Creating migration table ..... 70.65ms DONE
INFO Running migrations.
0001_01_01_000000_create_users_table ..... 86.60ms DONE
0001_01_01_000001_create_cache_table ..... 33.38ms DONE
0001_01_01_000002_create_jobs_table ..... 71.65ms DONE
2026_01_21_123404_create_libros_table ..... 16.29ms DONE
2026_01_21_123423_create_alumnos_table ..... 16.01ms DONE
2026_01_21_123447_create_prestamos_table ..... 46.92ms DONE

C:\Users\modor\Desktop\Final_DWES>docker-compose exec app php artisan make:model Libro
```

Formulario de Login

11.1.2. Gestión de Libros

Desde el menú lateral, seleccionar 'Libros':

- **Crear libro:** Hacer clic en el botón 'Nuevo Libro' y rellenar el formulario (ISBN, Título, Autor, Stock).

- **Editar libro:** En el listado, hacer clic en el icono de lápiz para modificar los datos.
- **Eliminar libro:** Hacer clic en el icono de papelera. Solo es posible si el libro no tiene préstamos activos.

```
C:\Windows\System32\cmd.exe
- Installing phar-io/version (3.2.1): Extracting archive
- Installing phar-io/manifest (2.0.4): Extracting archive
- Installing myclabs/deep-copy (1.11.0): Extracting archive
- Installing phpunit/phpunit (11.5.48): Extracting archive
61 package suggestions were added by new dependencies, use 'composer suggest' to see details.
Generating optimized autoload files
> illuminate\Foundation\ComposerScripts::postAutoloadDump
> @php artisan package:discover --ansi

[INFO] Discovering packages.

laravel/pail ..... DONE
laravel/sail ..... DONE
laravel/tinker ..... DONE
nesbot/carbon ..... DONE
nunomaduro/collision ..... DONE
nunomaduro/termwind ..... DONE

61 packages you are using are looking for funding.
Use the 'composer fund' command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force

[INFO] No publishable resources for tag [laravel-assets].

No security vulnerability advisories found.
> @php artisan key:generate --ansi

[INFO] Application key set successfully.

> @php -r "file_exists('database/database.sqlite') || touch('database/database.sqlite');"
> @php artisan migrate --graceful --ansi

[INFO] Preparing database.

Creating migration table ..... 222.78ms DONE

[INFO] Running migrations.

0001_01_01_000000_create_users_table ..... 255.55ms DONE
0001_01_01_000001_create_cache_table ..... 101.21ms DONE
0001_01_01_000002_create_jobs_table ..... 182.05ms DONE

C:\Users\modor\Desktop\Final_DWES>docker-compose exec app php artisan make:migration create_libros_table
```

Listado de libros con acciones

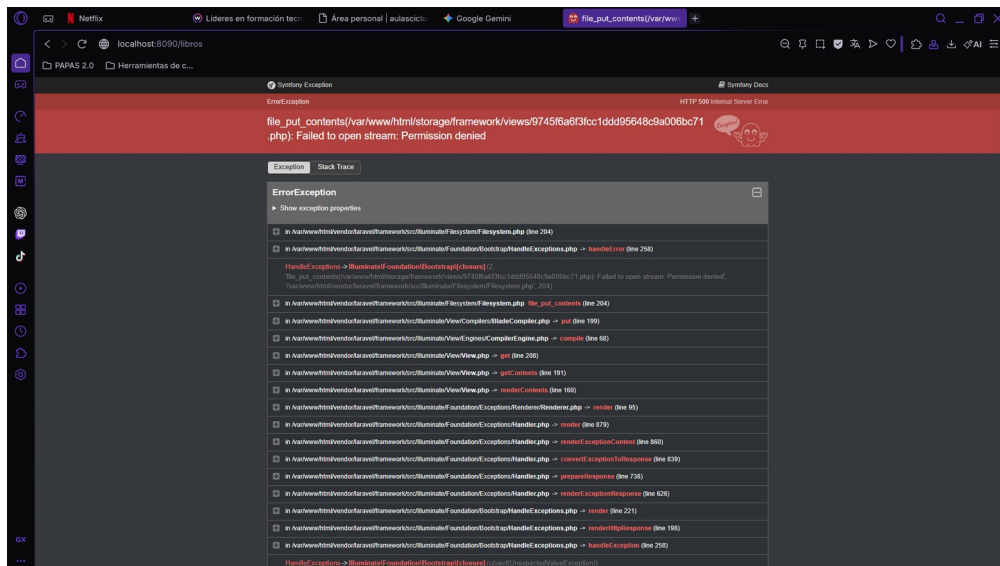
```
C:\Users\modor\Desktop\Final_DWES>docker-compose exec -e COMPOSER_PROCESS_TIMEOUT=2000 app composer create-project laravel/laravel .
time="2026-01-21T13:22:55.000" level="warning" msg="C:\Users\modor\Desktop\Final_DWES\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
Creating a "laravel/laravel" project at "."
Installing laravel/laravel (v12.11.2)
- Installing laravel/laravel (v12.11.2): Extracting archive
Created project in /var/www/html/.
> @php -r "file_exists('.env') || copy('.env.example', '.env');"
Loading composer repositories with package information
Updating dependencies
Lock file operations: 111 installs, 0 updates, 0 removals
- Locking brick/math (0.14.1)
- Locking carbonphp/carbon-doctrine-types (3.2.0)
- Locking dflydev/dot-access-data (v2.0.3)
- Locking doctrine/inflector (2.1.0)
- Locking doctrine/lexer (3.0.1)
- Locking dragonmantank/cron-expression (v3.6.0)
- Locking egulias/email-validator (4.0.4)
- Locking fakerphp/faker (v1.24.1)
- Locking filp/whoops (2.18.4)
- Locking fruitcake/php-cors (v1.4.0)
- Locking graham-campbell/result-type (v1.1.4)
- Locking guzzlehttp/guzzle (7.10.0)
- Locking guzzlehttp/promises (2.3.0)
- Locking guzzlehttp/psr7 (2.8.0)
- Locking guzzlehttp/uri-template (v1.0.5)
- Locking hamcrest/hamcrest-php (v2.3.1)
- Locking laravel/framework (v12.48.1)
- Locking laravel/pail (v1.2.0)
- Locking laravel/pint (v1.27.0)
- Locking laravel/prompts (v0.3.10)
- Locking laravel/sail (v1.52.0)
- Locking laravel/serializable-closure (v2.0.0)
- Locking laravel/tinker (v2.11.0)
- Locking league/commonmark (2.8.0)
- Locking league/config (v1.2.0)
- Locking league/flysystem (3.30.2)
```

Formulario de nuevo libro

11.1.3. Gestión de Alumnos

Desde el menú lateral, seleccionar 'Alumnos':

- **Registrar alumno:** Hacer clic en 'Nuevo Alumno' e introducir NIE, nombre, apellidos y email (opcional).
- **Editar alumno:** Usar el icono de edición para actualizar los datos del alumno.
- **Identificar morosos:** Los alumnos con el flag 'moroso' activado aparecen marcados en rojo.

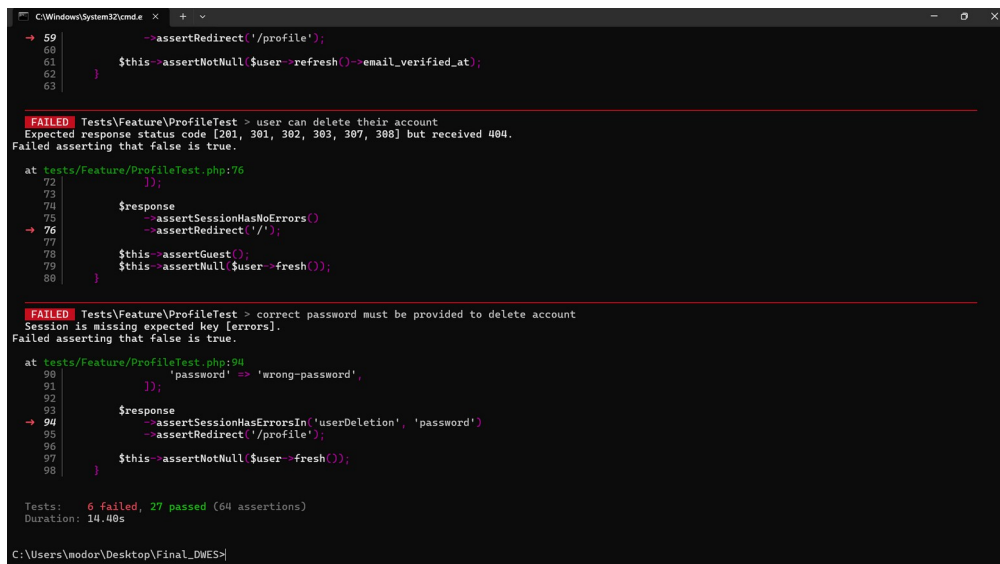


Listado de alumnos

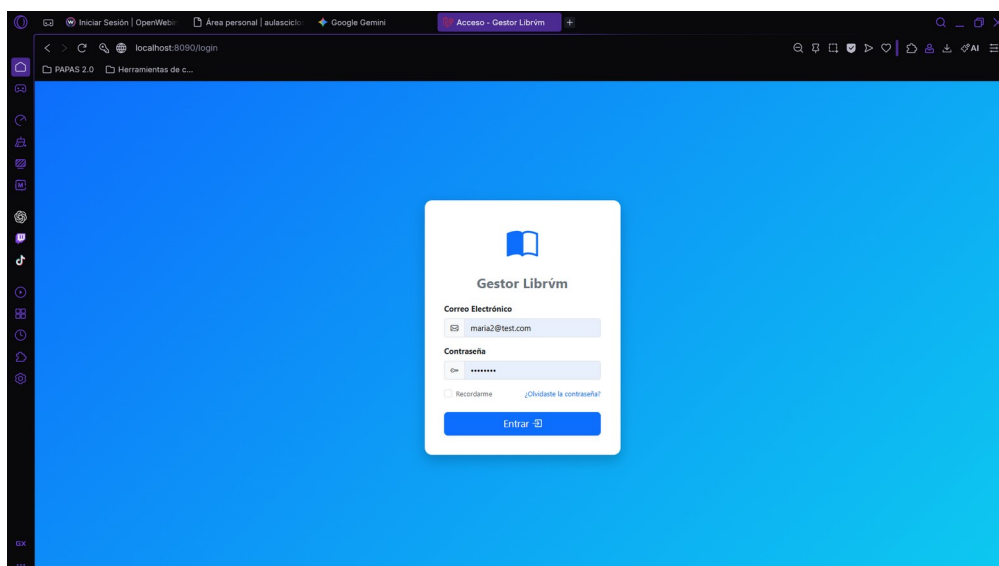
11.1.4. Gestión de Préstamos

Desde el menú lateral, seleccionar 'Préstamos':

- **Registrar préstamo:** Hacer clic en 'Nuevo Préstamo', seleccionar el libro y el alumno del desplegable. El sistema verifica automáticamente el stock antes de permitir el préstamo.
- **Registrar devolución:** En el listado de préstamos activos, hacer clic en 'Devolver'. El sistema registra la fecha y actualiza el stock.



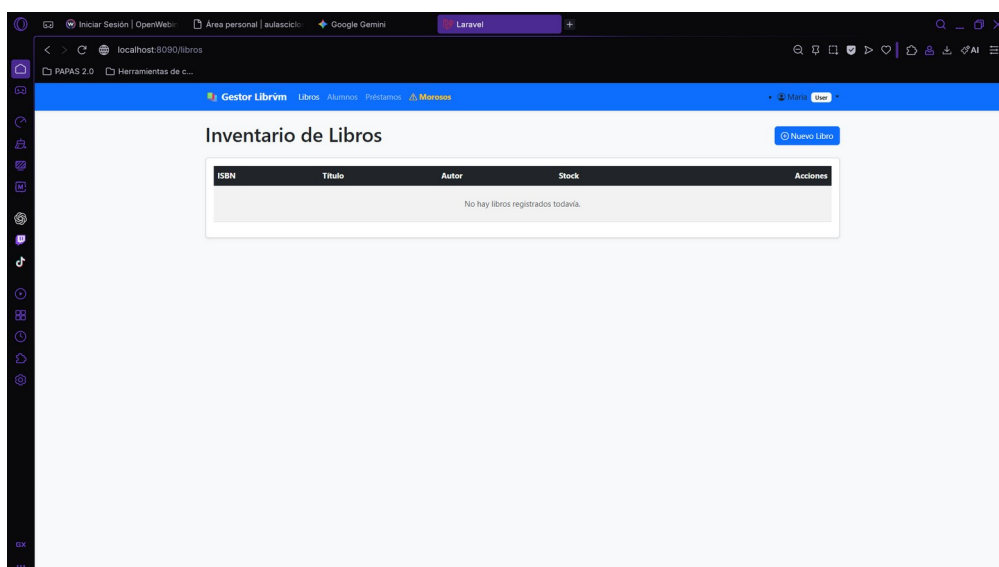
Listado de préstamos activos



Formulario de nuevo préstamo

11.1.5. Módulo de Morosos

Desde el menú lateral, seleccionar 'Morosos' para ver el listado de alumnos con préstamos vencidos o pendientes de devolución. Esta sección permite hacer seguimiento de los alumnos que no han devuelto los libros en el plazo establecido.



Listado de morosos

11.2. Manual para Administradores

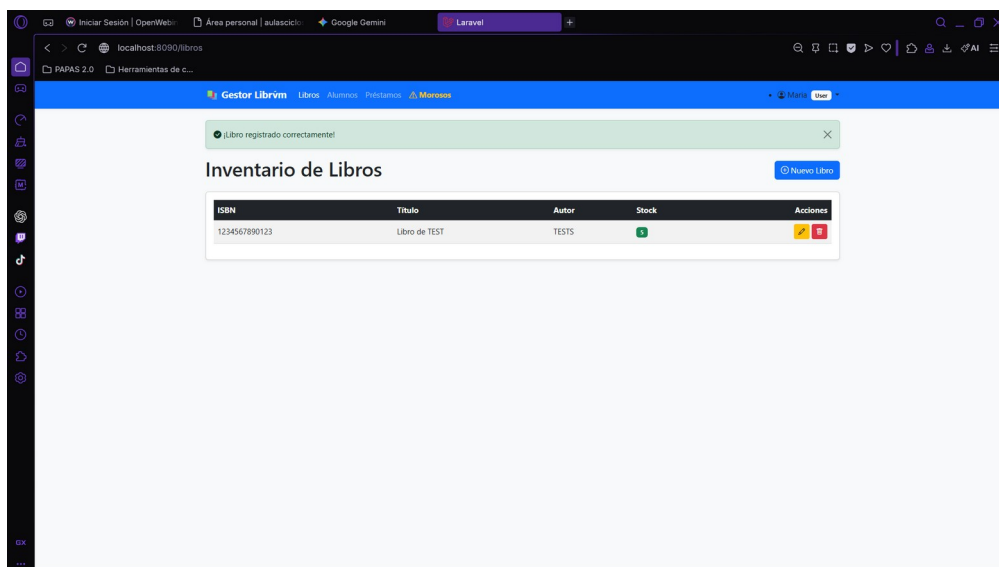
El administrador dispone de todas las funcionalidades del bibliotecario, más las siguientes secciones adicionales visibles en el menú:

11.2.1. Gestión de Usuarios del Sistema

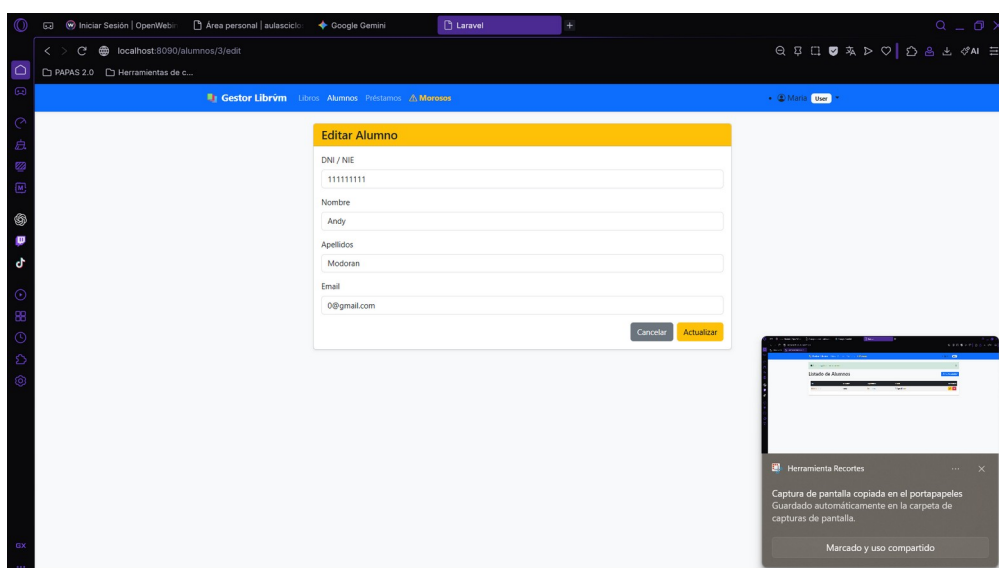
Desde 'Usuarios' en el menú, el administrador puede:

- **Crear usuario:** Hacer clic en 'Nuevo Usuario', introducir nombre, email, contraseña y seleccionar el rol (admin/user).
- **Editar usuario:** Modificar nombre, email o rol de cualquier usuario existente.

- **Eliminar usuario:** Eliminar usuarios que ya no deben tener acceso al sistema.



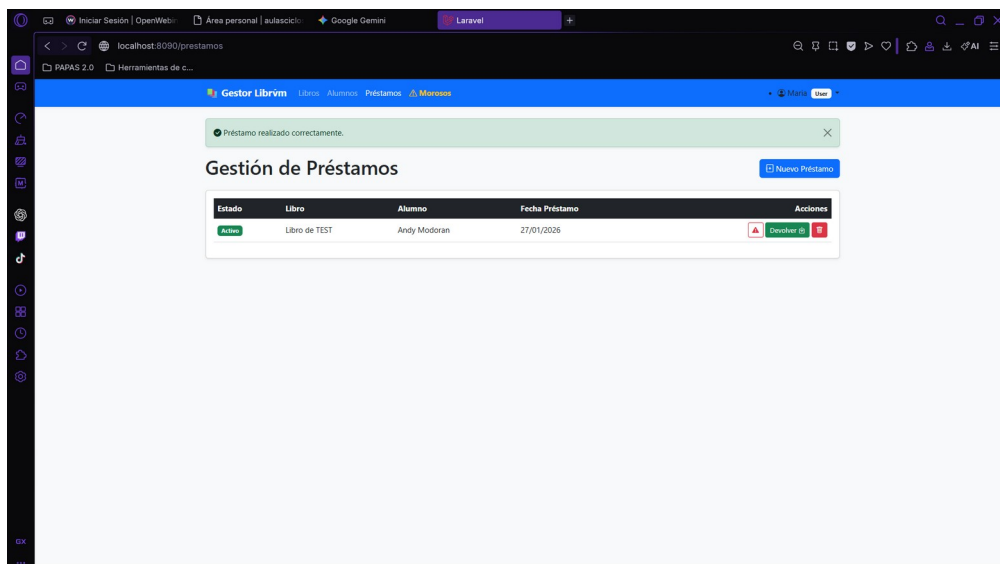
Panel de gestión de usuarios del sistema



Formulario de creación de usuario

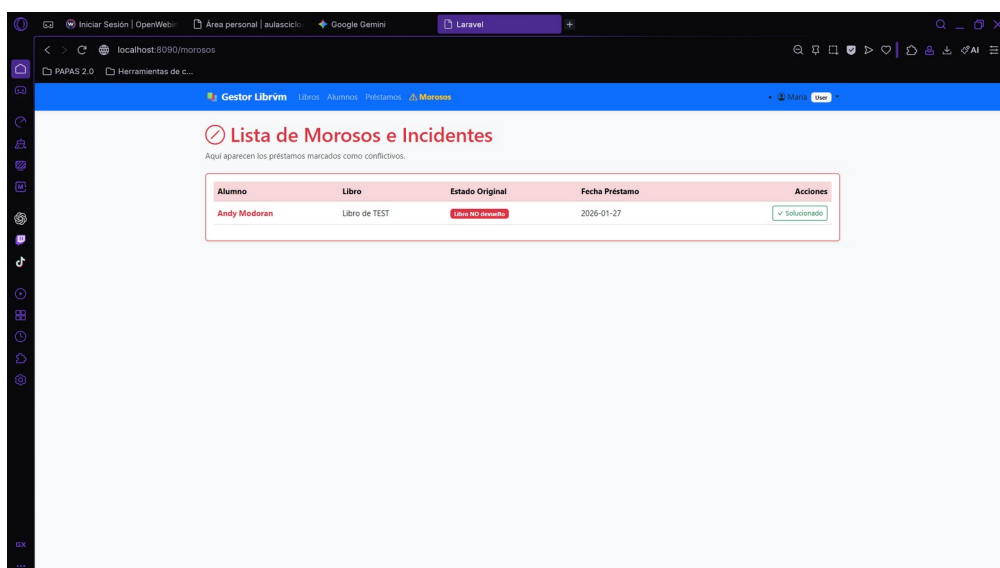
11.2.2. Auditoría — Historial de Movimientos

Desde 'Auditoría' o 'Movimientos' en el menú, el administrador visualiza un registro cronológico completo de todas las acciones críticas realizadas en el sistema: qué usuario realizó cada acción, qué tipo de operación fue y cuándo ocurrió exactamente.

*Historial de auditoría y movimientos del sistema*

11.2.3. Gestión del Perfil

Cualquier usuario (admin o bibliotecario) puede actualizar su nombre, email y contraseña desde la opción 'Mi Perfil' en el menú superior, accesible haciendo clic en el nombre del usuario.

*Página de edición del perfil de usuario*

12. CONCLUSIONES

12.1. Síntesis de Resultados

El desarrollo de Gestor Librvm ha permitido cumplir satisfactoriamente con todos los objetivos planteados inicialmente, resultando en una aplicación web funcional, segura y preparada para un entorno de producción real. Se destacan las siguientes conclusiones:

- **Robustez del Inventario:** La implementación de la lógica automática de stock en el PrestamoController ha sido un éxito. El sistema garantiza la integridad de los datos, impidiendo préstamos de libros agotados y asegurando que ningún libro se elimine si tiene un historial activo.
- **Seguridad Efectiva:** La segregación de roles (Admin vs Usuario) protege las áreas sensibles de la aplicación. Las pruebas automatizadas con PHPUnit confirman que el sistema es resistente a accesos no autorizados (403 en todos los casos verificados).
- **Calidad del Código:** La adopción de estándares como MVC y el uso de Docker han facilitado un desarrollo ordenado y reproducible. La batería completa de Tests Automatizados garantiza que el sistema sea mantenible y escalable sin riesgo de romper funcionalidades existentes.
- **Experiencia de Usuario:** La interfaz limpia basada en Bootstrap 5 permite que cualquier bibliotecario, independientemente de su nivel técnico, pueda utilizar la herramienta de forma intuitiva y eficiente.
- **Facilidad de Despliegue:** El script deploy.sh automatiza completamente el proceso de instalación, eliminando errores manuales y facilitando que cualquier persona pueda poner en marcha el sistema en cuestión de minutos.

12.2. Líneas Futuras de Trabajo

Aunque la aplicación es totalmente funcional en su versión actual (v1.0), se han identificado varias vías de mejora para versiones futuras:

19. **Sistema de Notificaciones por Email:** Integrar el sistema de correo de Laravel para enviar avisos automáticos a los alumnos cuando la fecha de devolución esté próxima o haya vencido.
20. **Escaneo de Códigos de Barras:** Implementar una librería JavaScript (como QuaggaJS) para permitir el escaneo del ISBN mediante la cámara del dispositivo, agilizando el proceso de préstamo.
21. **Gestión de Sanciones:** Ampliar el módulo de Morosos para calcular automáticamente una sanción (en días sin préstamo) basada en el retraso de la devolución.
22. **Carga de Imágenes:** Implementar el almacenamiento de archivos (Laravel Storage) para subir las portadas reales de los libros y las fotos de perfil de los usuarios.
23. **API REST:** Desarrollar una API REST pública para permitir, en el futuro, la creación de una aplicación móvil nativa para los alumnos.
24. **Optimización de Rendimiento:** Implementar paginación en todos los listados para mejorar el rendimiento con grandes volúmenes de datos.

BIBLIOGRAFÍA Y REFERENCIAS

- Documentación Oficial de Laravel: <https://laravel.com/docs>
- Documentación de Bootstrap 5: <https://getbootstrap.com/docs/5.3/>
- Docker y Docker Compose: <https://docs.docker.com/>
- PHPUnit Testing: <https://phpunit.de/>
- Eloquent ORM: <https://laravel.com/docs/eloquent>
- Laravel Breeze (Autenticación): <https://laravel.com/docs/starter-kits#laravel-breeze>
- Patron MVC: Design Patterns: Elements of Reusable Object-Oriented Software (Erich Gamma et al.)